

Day 6 Evidence Workbooklet

Branch Routing with if and else

Engineering practice → condition result → branch followed → selected action → support route

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/42		Mastery check	secure / developing / needs support	

The Week 2 scenario starts with a sensor-kit checkout table. A kit may be released, charged, sent to calibration, or held for missing checkout information. Today the focus is a single branch decision: read one condition, choose the branch that runs, and explain the action selected.

Engineering task	Decide which checkout action a sensor kit should receive from one condition at a time.
Computational move	Turn a true/false condition into one selected route while keeping the evidence for that route visible.
Python focus	<code>if</code> , <code>else</code> , condition result, selected branch, action variable, and printed route evidence.
Evidence to keep	case value, condition expression, <code>True/False</code> result, branch followed, action assigned, and support note.

Day 6 working rules

1. Python checks the condition after `if`.
2. If the condition is `True`, the `if` branch runs.
3. If the condition is `False`, the `else` branch runs.
4. Only one of those two branches runs.
5. A reliable branch note names both the condition result and the selected action.

1 Read the checkout condition before choosing an action

[recognize](#)[predict](#)

The lab team is checking whether a sensor kit can leave the checkout table. Day 6 uses one decision at a time: if the condition is true, one action happens; otherwise the other action happens.

```
1 kit_label = "S-14"
2 charge_percent = 82
3 charge_minimum = 80
4 calibration_label = "current"
5 borrower_initials = "JL"
6
7 charge_ok = charge_percent >= charge_minimum
8 calibration_ok = calibration_label == "current"
9 borrower_logged = borrower_initials != ""
10
11 if charge_ok:
12     action = "send to calibration check"
13 else:
14     action = "charge before checkout"
```

Pts.	Prompt	My evidence
1	Which variable stores the kit label?	
1	Which comparison creates <code>charge_ok</code> ?	
1	Is <code>charge_ok</code> true or false for this kit? Show the numbers.	
1	Which branch assigns the action for this kit?	

2 Trace an if/else route

[trace](#) [explain](#)

Complete the route table. Use the condition result to decide which branch runs.

Pts.	Charge value	Condition	Branch followed	Action and reason
2	82	<code>charge_percent</code> <code>>= 80</code>		
2	80	<code>charge_percent</code> <code>>= 80</code>		
2	79	<code>charge_percent</code> <code>>= 80</code>		
2	Explain why the exact value 80 matters.			

Boundary connection

The route depends on a comparison from Week 1. If the comparison is wrong, the branch route will be wrong too.

3 Choose the selected action from printed evidence

[predict](#)[explain](#)

The same structure is used for calibration status. Read the code and predict the action.

```
1 calibration_label = "expired"
2 calibration_ok = calibration_label == "current"
3
4 if calibration_ok:
5     checkout_status = "ready for borrower log"
6 else:
7     checkout_status = "send to calibration bench"
8
9 print(kit_label, checkout_status)
```

Pts.	Prompt	My evidence
2	What value is stored in <code>calibration_ok</code> ? Why?	
2	Which branch runs: the <code>if</code> branch or the <code>else</code> branch?	
2	What exact status text is printed after the kit label?	
2	Why would <code>bool(calibration_label)</code> be unsafe here?	

4 Complete one branch condition

[implement](#) [debug](#)

Complete the condition so the kit moves to borrower logging only when the borrower initials are present.

```
if _____:
    next_step = "record borrower and release kit"
else:
    next_step = "hold until borrower initials are entered"
```

Pts.	Prompt	My response
3	Write the completed condition using <code>borrower_initials</code> .	
2	Why should the condition compare against an empty string instead of checking the kit label?	
2	Give one input value that should follow the <code>else</code> branch.	
1	Name the evidence type you used: state / type / boundary / branch.	

5 Debug a branch-routing claim

[debug](#) [test](#) [explain](#)

A teammate writes: "The kit is ready because `charge_ok` is true." Use branch-routing evidence to evaluate that claim.

Pts.	Prompt	My response
2	What part of the claim is supported by the code?	
2	What evidence is still missing before a full checkout release?	
2	Give one test case that should follow the hold/repair route.	
2	Write a safer one-sentence checkout note.	

Pts.	My checkout-routing note
4	Write a short note that says how an if/else route chooses one action from one condition. Use the sensor-kit checkout example.
2	Circle the support route you would use next: condition result / branch followed / exact boundary / string label / written explanation. Name one next action: _____

Bridge to Day 7

Day 6 chooses between two actions. Day 7 adds ordered rules where the first true condition wins and later branches are skipped.

Scope boundary: use the wrapper; do not author it

The `def` line is provided in the notebook. Do not rewrite it or rename its parameters. Read the parameter names as inputs, follow the indented body, and use the returned value. This page adds no points; the workbook score remains unchanged.

```
def badge_sheet_first_route(group_count, badges_per_group):
    total_badges = group_count * badges_per_group
    if total_badges <= 8:
        return "one sheet"
    else:
        return "more than one sheet"
```

Function call	Total badges	Returned text and route evidence
badge_sheet_first_route(4, 2)		
badge_sheet_first_route(3, 3)		
badge_sheet_first_route(2, 4)		

Notebook connection

VIP Module 18 Activity 18.8 uses the same two inputs and calculation but eventually needs three routes. Today, identify the fixed header, the parameter values, the intermediate quantity, and the returned text. Day 7 adds the middle route.